

Ownership, the Approved List and Trust

The browser application would be one permanent shell that many feature plugins draw inside. This sheet says who owns which part, what the platform's approved list of plugins holds, and what an entry must pass before the shell will count it. **Maturity is marked per element:** a solid tile is built and running in the Dictionary POC today, a dashed grey tile is a **directional** V3 position as at 4 September 2026 with no decision of record, and a dashed amber tile needs a named owner. Subjects and grants belong to LB-V3-L3-05; the sign-in token belongs to LB-V3-L3-04; where the files are served belongs to LB-V3-L3-08; field-by-field shapes belong to L4.

WHO OWNS WHICH PART OF THE APPLICATION

F Application frame

one shell, one page
topbar, menu, content, footer
built in the lab

G Global providers

routing, state, theme
the shell installs them once
built in the lab

D Plugin discovery

an approved list only
never self-registration
built in the lab

S Contribution slots

the host screen owns them
the host picks the place
built in the lab

P Feature code

the plugin owns it
its own screens and state
built in the lab

the approved list is the only thing that may nominate code

THE APPROVED LIST OF PLUGINS

1 One entry per plugin

name, version, place, contributions
the description, not the code
built in the lab

2 Kinds kept apart

routes, menu entries, named slots
each kind is checked on its own
built in the lab

3 A stated fit

the plugin says which shell it needs
outside that range it is refused
built in the lab

4 Asked for at run time

the platform answers with the list
not a file inside the bundle
built in the lab

an entry no operator typed must pass three gates

THREE GATES BEFORE AN ENTRY COUNTS

1 · Ownership

does this publisher own the name?
a wrong name is its own failure
built in the lab

2 · Signature

signed by a key trusted right now
a withdrawn key stops verifying
built in the lab

3 · Release number

higher than the last one accepted
the same number is a safe repeat
built in the lab

L Where code may come from

an approved list of locations
the gates do not read the code
built in the lab

an accepted plugin still shows only what you may see

PERMISSION AND IDENTITY

K Permission

read from the sign-in token
one source for every plugin
built in the lab

! Hidden is not blocked

the service still refuses
a hidden button is a courtesy
built in the lab

C Separate connections

each plugin keeps its own
convenience never widens rights
built in the lab

Q Nothing secret is written

no tokens, keys or message content
not in the list, loader or logs
built in the lab

X A signature says who, never what

trusted code that turns hostile is still trusted

where code may come from is bounded by the location list
built in the lab · the fetched code itself is unsigned

? Who holds the publisher keys in V3?

no named owner yet to issue, enable or withdraw

open item L3-03-001 · platform engineering and security
no V3 decision of record as at 2026-09-04

Read it downward. The shell owns the frame, an approved list says which plugins exist, three gates decide whether an entry counts, and permission decides what a person then sees. **Built today:** every solid tile — the shell, the list, the three gates and the permission source. **Not shown:** subjects and grants, which are LB-V3-L3-05 ; the sign-in token itself, which is LB-V3-L3-04 ; entry fields and contribution shapes, which are L4 .

LEGEND

 Cyan boundary — who owns which part of the application

 Solid tile — built and running in the lab today

 Red panel — a limit that must not be misread

► **Arrow head** — the direction of the relationship

 Green boundary — the platform's approved list of plugins

 Dashed grey tile — proposed for V3, not built

→ **Green line** — ownership leading to the approved list

Line label — names the relationship, never bare "uses"

 Blue boundary — the checks an entry must pass

 Dashed amber tile — open, a named owner must confirm

→ **Blue line** — a list entry entering the three gates

 Amber boundary — permission and identity

 Amber panel — an open question that governs the sheet

- - - → **Amber dashed line** — an accepted plugin reaching permission

WORDS ON THIS SHEET

shell the permanent frame every plugin draws in
plugin one feature, built and shipped on its own
MFE micro front end, another word for a plugin
approved list the platform's list of allowed plugins
entry one plugin's description on that list
contribution a route, menu entry or slot a plugin fills

slot a place a screen offers others to draw in
route a web address inside the application
publisher whoever signs and offers an entry
release number a counter that only goes up
sign-in token proof of who a person is
ADR architecture decision record

SCOPE	One question. When is a plugin's code fetched, what can break, and what is a working plugin given? Ownership is on sheet 1.
RULE	Description before code. The shell must know what a plugin claims to be before it runs a single line of it.
LAZY	Menus and links are live at step 3. Declaring a link does not download the feature behind it.
ONCE	A plugin starts once per version. A second visit to its screen reuses what is already running.
STATES	Refused and broken are different. One means we would not take it; the other means we tried and it failed.
BLAST	Contentment is structural, not a nicer error message. A failed start is rolled back , not left half done.
SLOT	The host names the slot, its shape, its order and how many fit. The plugin fills it and nothing else.
SPLIT	Language belongs to the person; content belongs to the connection. That split is what lets language stay shell-wide.
L3-07	Labels, translations and theme values are stored by the data view. This sheet only shows them being used.
OPEN	The connection broker, the supported version window and mid-session disabling all need a named owner.

- **Cyan boundary** — the ordered steps before code runs
- **Solid tile** — built and running in the lab today
- **Grey line on a tile** — that element's maturity
- **Arrow head** — the direction of the relationship

-  **Blue boundary** — how a failure is contained
-  **Dashed amber tile** — open, a named owner must confirm
-  **Blue line** — a state reaching the failure it can enter

-  **Amber boundary** — what a plugin is handed, and what is refused
-  **Numbered tile** — the order to read a band in
-  **Amber dashed line** — a surviving plugin reaching what it is handed

boot the shell starting up in the browser	refused we would not take it; no code ran
index filing entries so menus and links work	broken we tried it and it failed
first use the first time a link or slot is opened	roll back undo a part-finished install
fetch downloading a plugin's code	slot a place a screen offers others to draw in
start running a plugin's set-up once	reference data shared labels, values and theme
state where one plugin has got to right now	broker one shared holder of connections